

## Question Paper Set A1

AI-Assisted Coding Exam — Python (1 hour)

- You may either (a) write the exact prompt to have your AI assistant generate code, or (b) write the code yourself. If you use AI, submit the prompt and the final code you executed.
- Use VS Code Copilot / Gemini / CursorAI. Credit your prompts.
- Prefer Python standard library; write clear, tested, well-documented code.
- Each subgroup (A–O) has two tasks. Attempt all. Medium complexity, use-case based.
- Syllabus pillars:
  - A–C: AI completion (classes, loops, conditionals)
  - D–E: Debugging with AI
  - F–G: Code Quality & Performance
  - H–I: Documentation & Code Review
  - J–K: Code Quality & Performance
  - L–M: Debugging with AI
  - N–O: AI completion (classes, loops, conditionals)

Deliverables for each question

- 1) If using AI: the exact prompt you issued. If manual: note 'manual' and a brief design reason.
- 2) solution.py
- 3) tests.py (unittest/pytest; write tests first for TDD items)
- 4) Docstrings & inline comments (AI-assisted allowed)
- 5) Short README.md (approach, assumptions, complexity, run tests)
- 6) For debugging/refactor: brief before/after note

## Question Paper Set A1

AI-Assisted Coding Exam — Python (1 hour)

- You may either (a) write the exact prompt to have your AI assistant generate code, or (b) write the code yourself. If you use AI, submit the prompt and the final code you executed.
- Use VS Code Copilot / Gemini / CursorAI. Credit your prompts.
- Prefer Python standard library; write clear, tested, well-documented code.
- Each subgroup (A–O) has two tasks. Attempt all. Medium complexity, use-case based.

- Syllabus pillars:

A–C: AI completion (classes, loops, conditionals)

D–E: Debugging with AI

F–G: TDD with AI

H–I: Documentation & Code Review

J–K: Code Quality & Performance

L–M: Files/CSV & Regex

N–O: AI completion (classes, loops, conditionals)

Deliverables for each question

1) If using AI: the exact prompt you issued. If manual: note 'manual' and a brief design reason.

2) solution.py

3) tests.py (unittest/pytest; write tests first for TDD items)

4) Docstrings & inline comments (AI-assisted allowed)

5) Short README.md (approach, assumptions, complexity, run tests)

6) For debugging/refactor: brief before/after note

## Subgroup I

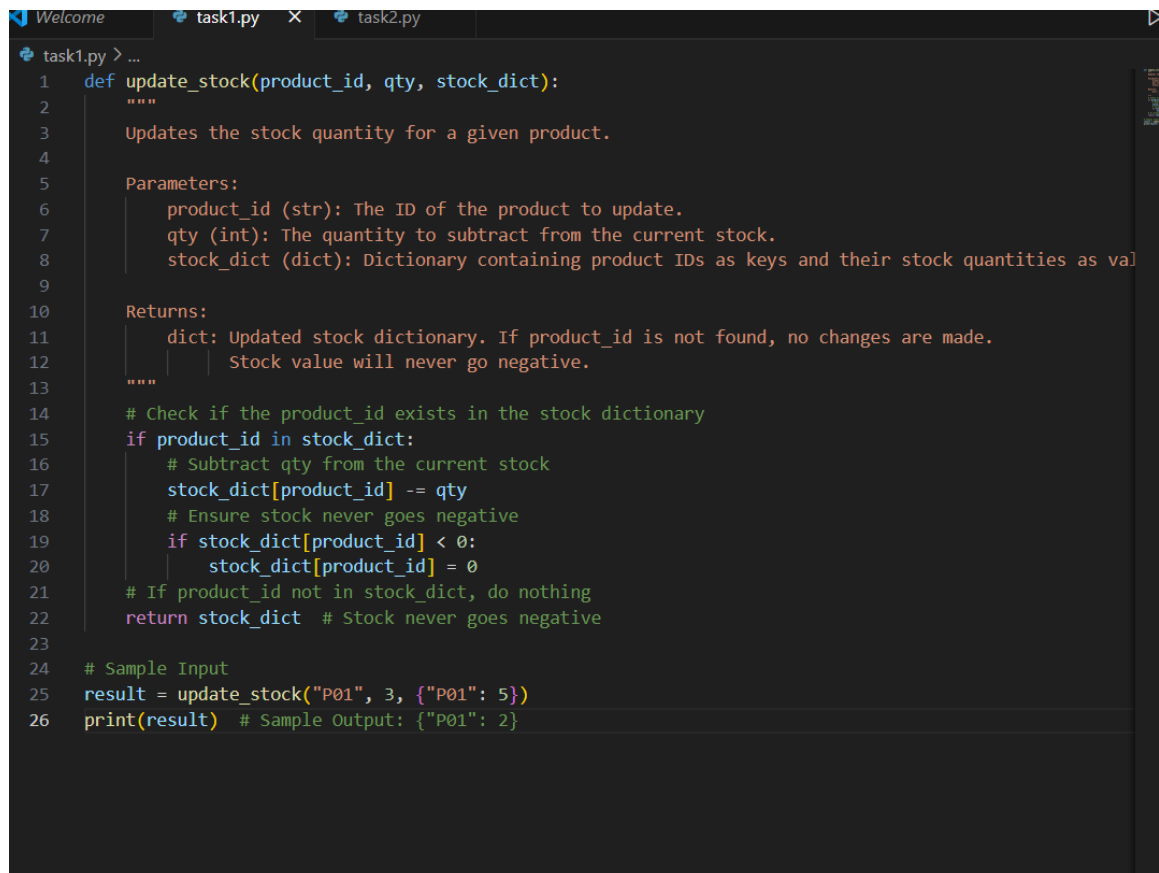
### I.1 — [S01I1] Generate Documentation for Inventory Management

#### Scenario (e-commerce):

A junior developer wrote a Python function that updates product stock after an order is placed. However, the function lacks documentation, making it hard to maintain.

```
def update_stock(product_id, qty, stock_dict):  
    if product_id in stock_dict:  
        stock_dict[product_id] -= qty  
        if stock_dict[product_id] < 0:  
            stock_dict[product_id] = 0  
    return stock_dict
```

#### DEBUGGING CODE:

A screenshot of a code editor window with three tabs: 'Welcome', 'task1.py', and 'task2.py'. The 'task1.py' tab is active, showing a Python function 'update\_stock' with comprehensive documentation. The code is color-coded: keywords in blue, strings in orange, comments in green, and docstrings in orange. The function includes a docstring with a description, parameters, and returns. It also has inline comments and a sample input/output at the bottom.

```
task1.py > ...  
1  def update_stock(product_id, qty, stock_dict):  
2      """  
3          Updates the stock quantity for a given product.  
4  
5          Parameters:  
6              product_id (str): The ID of the product to update.  
7              qty (int): The quantity to subtract from the current stock.  
8              stock_dict (dict): Dictionary containing product IDs as keys and their stock quantities as values.  
9  
10         Returns:  
11             dict: Updated stock dictionary. If product_id is not found, no changes are made.  
12             Stock value will never go negative.  
13         """  
14         # Check if the product_id exists in the stock dictionary  
15         if product_id in stock_dict:  
16             # Subtract qty from the current stock  
17             stock_dict[product_id] -= qty  
18             # Ensure stock never goes negative  
19             if stock_dict[product_id] < 0:  
20                 stock_dict[product_id] = 0  
21         # If product_id not in stock_dict, do nothing  
22         return stock_dict # Stock never goes negative  
23  
24         # Sample Input  
25         result = update_stock("P01", 3, {"P01": 5})  
26         print(result) # Sample Output: {"P01": 2}
```

#### OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\supri\OneDrive\Desktop\AIAC\labexam2> & C:/Users/supri/AppData/Local
supri/OneDrive/Desktop/AIAC/labexam2/task1.py
{'P01': 2}
PS C:\Users\supri\OneDrive\Desktop\AIAC\labexam2>
```

### Your Task:

- Use AI to generate **comprehensive docstrings and inline comments**.
- Ensure the docstring specifies parameters, return type, and edge cases (like negative stock).

### Data & Edge Cases:

- Stock must never go negative.
- If `product_id` not in stock → no update.

### AI Assistance Expectation:

- Use AI to propose a PEP 257-style docstring.
- AI should add inline comments explaining **why** conditions exist, not just **what** they do.

### Sample Input

```
update_stock("P01", 3, {"P01": 5})
```

### Sample Output

```
{"P01": 2}
```

### Acceptance Criteria:

- Function retains same logic.
- Generated documentation is clear, professional, and AI-assisted.
- Inline comments add readability without redundant

---

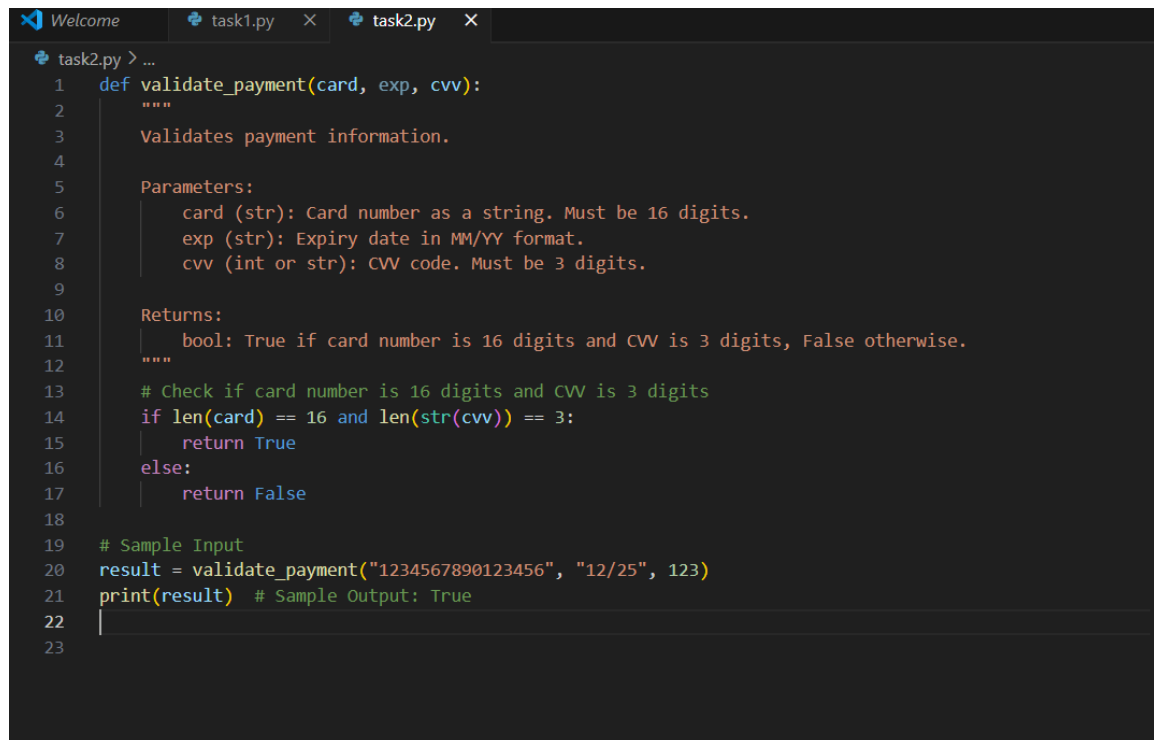
## I.2 — [S01I2] AI-Assisted Code Review for Payment Validation

### Scenario (fintech/e-commerce):

A developer wrote the following function to validate payment details. It works but may not meet quality or security standards.

```
def validate_payment(card, exp, cvv):  
    if len(card) == 16 and len(str(cvv)) == 3:  
        return True  
    else:  
        return False
```

### DEBUGGING CODE:



```
Welcome task1.py task2.py  
task2.py > ...  
1 def validate_payment(card, exp, cvv):  
2     """  
3     Validates payment information.  
4  
5     Parameters:  
6     card (str): Card number as a string. Must be 16 digits.  
7     exp (str): Expiry date in MM/YY format.  
8     cvv (int or str): CVV code. Must be 3 digits.  
9  
10    Returns:  
11    | bool: True if card number is 16 digits and CVV is 3 digits, False otherwise.  
12    """  
13    # Check if card number is 16 digits and CVV is 3 digits  
14    if len(card) == 16 and len(str(cvv)) == 3:  
15        return True  
16    else:  
17        return False  
18  
19    # Sample Input  
20    result = validate_payment("1234567890123456", "12/25", 123)  
21    print(result) # Sample Output: True  
22  
23
```

### OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\supri\OneDrive\Desktop\AIAC\labexam2> & C:/Users/supri/AppData/Local/Pr
supri/OneDrive/Desktop/AIAC/labexam2/task2.py
True
PS C:\Users\supri\OneDrive\Desktop\AIAC\labexam2>
```

### Your Task:

- Perform an **AI-assisted code review**.
- Suggest improvements regarding:
  - Readability
  - Code quality
  - Security best practices (don't log sensitive data, avoid weak checks).
- Rewrite the function with improvements, using AI to scaffold.

### Data & Edge Cases:

- Expiry date validation missing.
- `card` type not enforced (should be digits only).
- Should gracefully handle invalid inputs (e.g., `None`, wrong length).

### AI Assistance Expectation:

- Ask AI for review suggestions.
- Apply fixes (e.g., regex for card digits, structured return values).

### Sample Input

```
validate_payment("1234567890123456", "12/25", 123)
```

### Sample Output

```
True
```

### Acceptance Criteria:

- Reviewed code has improved readability & error handling.
- AI-proposed improvements are documented (before → after).

- No sensitive data exposed in logs or messages.

-----